

Declaración responsable sobre autoría y uso ético de inteligencia artificial, otros recursos o ayuda externa

Con la entrega de esta tarea, declaro de manera responsable que es el resultado de mi trabajo intelectual personal y creativo, y que ha sido elaborado de acuerdo con los principios éticos y las normas de integridad vigentes en la comunidad académica y, más específicamente, en la [Universidad Complutense de Madrid](#).

Soy, pues, autor del material aquí incluido y, cuando no ha sido así y he tomado el material de otra fuente, lo he citado o bien he declarado su procedencia de forma clara - incluidas, en su caso, herramientas de inteligencia artificial-. Las ideas y aportaciones principales incluidas en este trabajo, y que acreditan la adquisición de competencias, son mías y no proceden de otras fuentes o han sido reescritas usando material de otras fuentes.

Antes de entregar esta tarea, asegúrate de que todo funcione como se espera:

1. Reinicia el kernel

(En la barra de menú, selecciona **Kernel** → **Restart**)

2. Ejecuta todas las celdas

(En la barra de menú, selecciona **Cell** → **Run All**)

3. Asegúrate de completar todos los lugares donde aparezca:

- YOUR CODE HERE
- "YOUR ANSWER HERE"

4. Debes borrar las instrucciones que pone `raise NotImplementedError()`

5. Así como tu **nombre** a continuación:

```
In [1]: NOMBRE = "Pablo Blas Hernández"
```

Notas para la entrega

- Se deben rellenar todos los cuadernos sin cambiar el nombre.
- Se debe comprimir la carpeta en un fichero `.zip`, llamado `tarea_final.zip`.
Importante: por favor, no se entreguen ficheros `.rar`.
- Subir el fichero comprimido al campus virtual.

Agencia para el Empleo del ayuntamiento de Madrid. Perfiles de personas inscritas

En este trabajo vamos a trabajar con datos del portal de datos abiertos del ayuntamiento de Madrid. En particular usaremos datos de la [Agencia de empleo](#).

Citando la propia página:

"La Agencia para el Empleo de Madrid es la entidad encargada por el Ayuntamiento de Madrid para la gestión de políticas activas de empleo, al objeto de facilitar y potenciar la inserción laboral de las personas sin empleo de la Ciudad de Madrid."

"La Agencia actúa como una plataforma multilateral en la que reúne a dos grupos distintos pero interdependientes, los trabajadores/as desempleados/as y las empresas."

"Este conjunto de datos ofrece la relación de perfiles de personas inscritas en la Agencia para el Empleo para poder acceder a sus servicios. Se incluye, entre otros datos, el detalle de la fecha de inscripción, sexo, año de nacimiento, nacionalidad y el objetivo profesional en cada caso. "

Junto a este enunciado hay un fichero con formato CSV, `inscripciones.csv`, con los datos de las personas inscritas en lo que se lleva de año 2025. Es un fichero CSV cuyo separador es el `;` y los datos necesarios están marcados con `"`. Podemos visualizar las 5 primeras líneas ejecutando la siguiente casilla:

```
In [2]: DATAFILE = 'inscritos.csv'
TESTDATAFILE = 'inscritos_test_sample.csv'
with open(DATAFILE, encoding='utf-8-sig') as f:
    for _ in range(5):
        print(f.readline(), end='')
```

```
FECHA_INSCRIPCION;GENERO_DESC;DISTRITO_COD;DISTRITO_DESC;EDAD;NACIONALIDAD_DESC;OBJETIVOPROFESIONAL1_COD;OBJETIVOPROFESIONAL1_DESC;OBJETIVOPROFESIONAL2_COD;OBJETIVOPROFESIONAL2_DESC;OBJETIVOPROFESIONAL3_COD;OBJETIVOPROFESIONAL3_DESC;FX_CARGA
"ene-24";"Hombre";10;" LATINA";34;"Español";"5833";"Conserjes de edificios";"5220";"Vendedores en tiendas y almacenes";"9602";"Peones de la construcción de edificios";"2025-08-07 00:11:57.897"
"ene-20";"Hombre";10;" LATINA";40;"Extracomunitario";"5500";"Cajeros y taquilleros (excepto bancos)";"5220";"Vendedores en tiendas y almacenes";"9602";"Peones de la construcción de edificios";"2025-08-07 00:11:57.897"
"nov-23";"Hombre";13;" PUENTE DE VALLECAS";24;"Extracomunitario";"3510";"Agentes y representantes comerciales";"5420";"Operadores de telemarketing";"9602";"Peones de la construcción de edificios";"2025-08-07 00:11:57.897"
"oct-23";"Hombre";17;" VILLAVERDE";19;"Extracomunitario";"9820";"Reponedores";"5220";"Vendedores en tiendas y almacenes";"9602";"Peones de la construcción de edificios";"2025-08-07 00:11:57.897"
```

Para comprobar la correcta definición de los tipos, necesitamos el módulo `pytest`, que se instala ejecutando la siguiente línea:

```
In [3]: !pip install pytest
```

Requirement already satisfied: pytest in c:\users\pabli\anaconda3\lib\site-packages (7.1.2)
 Requirement already satisfied: attrs>=19.2.0 in c:\users\pabli\anaconda3\lib\site-packages (from pytest) (21.4.0)
 Requirement already satisfied: iniconfig in c:\users\pabli\anaconda3\lib\site-packages (from pytest) (1.1.1)
 Requirement already satisfied: packaging in c:\users\pabli\anaconda3\lib\site-packages (from pytest) (21.3)
 Requirement already satisfied: pluggy<2.0,>=0.12 in c:\users\pabli\anaconda3\lib\site-packages (from pytest) (1.0.0)
 Requirement already satisfied: py>=1.8.2 in c:\users\pabli\anaconda3\lib\site-packages (from pytest) (1.11.0)
 Requirement already satisfied: tomli>=1.0.0 in c:\users\pabli\anaconda3\lib\site-packages (from pytest) (2.0.1)
 Requirement already satisfied: atomicwrites>=1.0 in c:\users\pabli\anaconda3\lib\site-packages (from pytest) (1.4.0)
 Requirement already satisfied: colorama in c:\users\pabli\anaconda3\lib\site-packages (from pytest) (0.4.5)
 Requirement already satisfied: pyparsing!=3.0.5,>=2.0.2 in c:\users\pabli\anaconda3\lib\site-packages (from packaging->pytest) (3.0.9)

En todo proyecto se ha de incluir la importación de librerías y la definición de constantes en su inicio. Acompañando a este enunciado, dispones de un fichero llamado `testing.py` con funciones de prueba que realizan comprobaciones en tus funciones. También se debe cargar este fichero, y otros necesarios, como puedes ver en la siguiente celda.

También, el fichero es bastante grande, unos 30Mb. Para que sea más sencillo desarrollar el programa adjuntamos también un fichero de muestra con el mismo formato, pero con un tamaño más manejable para hacer pruebas: `inscritos_test_sample.csv`. Te aconsejamos que pruebes tus funciones con este fichero más pequeño antes de probar con el fichero más grande.

```
In [4]: import pytest
import doctest
from typing import TypedDict
import locale
import os

# Importaciones
import csv      # Procesamiento de archivos csv
import random   # Muestreo aleatorio
from datetime import datetime  # Manejo de fechas temporales
from collections import Counter, defaultdict  # Estructuras de datos eficientes
import testing
from typing import Tuple
import numpy as np  # Para realizar los ejercicios de numpy

# Las constantes que defines deben ir aquí detrás:
# Archivos de datos
DATAFILE = 'inscritos.csv'  # Dataset completo
TESTDATAFILE = 'inscritos_test_sample.csv'  # Muestra pequeña para testing rápida
```

En este enunciado encontrarás casillas parecidas a estas. Sirven para comprobar si las funciones que defines funcionan como se espera de ellas. Si dan algún tipo de error es

que tienes algo que corregir. La casilla que tienes simplemente comprueba que los ficheros de datos están correctamente en su sitio.

```
In [5]: testing.test_datafile(DATAFILE, TESTDATAFILE)
```

OK. Los ficheros están en su sitio. Puedes continuar

Muestreo aleatorio

Define una función llamada `gen_sample` que tenga como datos el nombre de un fichero (un `str`) y un `float` entre 0 y 1 que indica la probabilidad con que cada línea del fichero original va a ser seleccionada en el set de muestra. Para ello debes usar la función `random` del módulo `random`, que genera de forma aleatoria un número aleatorio de forma uniforme entre 0 y 1. El fichero de muestra generado debe nombrarse como el original, añadiendo un sufijo al nombre del tipo `_XX` antes de la extensión. `XX` es la probabilidad indicada, (el número que, multiplicado por 100, dará aproximadamente el porcentaje de líneas escogidas) con una precisión de 2 dígitos. Por ejemplo, si el fichero es `inscritos.csv` y la probabilidad 0.05 (eo sea, se seleccionarán aproximadamente el 5% de los datos pra la muestra), el fichero generado se debe llamar `inscritos_05.csv`.

El primer argumento de la función es el fichero a procesar y el segundo, el porcentaje. La función debe comprobar que el argumento del porcentaje está entre 0 y 1.

La función debe ir leyendo el fichero. Para cada línea, se debe generar un número aleatorio. Si el número generado es inferior al porcentaje, esa línea se incluye en el fichero destino. Este método no garantiza un porcentaje exacto, pero sí proporciona una aproximación aceptable, teniendo en cuenta el gran tamaño del conjunto de datos de prueba.

La línea primera, del encabezado, se debe incluir en cualquier caso.

```
In [6]: def gen_sample(file_name: str, p: float) -> None:
        """
        Genera un archivo de muestra seleccionando líneas aleatorias del archivo.

        Esta función crea subconjuntos de datos más manejables cuando trabajamos con

        Args:
            file_name: archivo csv original
            p: probabilidad de selección (0 < p < 1)

        Returns:
            None. Crea un archivo en disco con el nombre modificado de salida genera
            (datos_05.csv en caso de p= 0.05, datos_01 si p= 0.01, etc.)

        Raises:
            Error si p no está en el rango (0,1)

        Examples:
            # Prueba de excepción para porcentaje fuera de rango
            >>> try:
```

```

...     gen_sample("dummy.csv", 1.5)
... except ValueError as e:
...     print(e)
La probabilidad debe estar entre 0 y 1.

# Prueba del nombre de archivo de salida
>>> base, ext = os.path.splitext("inscritos.csv")
>>> p_test = 0.05
>>> suffix = f"_{int(p_test * 100):02d}"
>>> out_name = f"{base}{suffix}{ext}"
>>> out_name
'inscritos_05.csv'
"""
# Validación del porcentaje
if not (0 < p < 1):
    raise ValueError("La probabilidad debe estar entre 0 y 1.")

# Obtenemos el nombre base y su extensión
base, ext = os.path.splitext(file_name)
# Creamos el sufijo del porcentaje (dos dígitos)
suffix = f"_{int(p * 100):02d}"
# Nombre de salida
out_name = f"{base}{suffix}{ext}"

with open(file_name, "r", encoding="utf-8") as fin, \
    open(out_name, "w", encoding="utf-8") as fout:

    header = fin.readline()
    fout.write(header)

    # Cada línea se incluye con probabilidad p para generar el número
    for line in fin:
        if random.random() < p: #random() genera un número uniforme en [0,1
            fout.write(line)

return None

```

```

In [7]: # En esta casilla, puedes generar tus propias pruebas.

# Prueba 1: Excepción por p inválido
try:
    gen_sample("dummy.csv", 1.5)
except ValueError as e:
    print(e)    # Debe imprimir: "La probabilidad debe estar entre 0 y 1."

# Prueba 2: Generación del nombre de archivo
base, ext = os.path.splitext("inscritos.csv")
p_test = 0.05
suffix = f"_{int(p_test * 100):02d}"
out_name = f"{base}{suffix}{ext}"

print(out_name)    # Debe imprimir: inscritos_05.csv

```

La probabilidad debe estar entre 0 y 1.
 inscritos_05.csv

Autocomprobación

```
In [8]: def check_gen_sample_result():
import os
gen_sample(DATAFILE, 0.01)
data01 = DATAFILE.replace('.csv', '_01.csv')
with open(data01, encoding='utf-8-sig') as f:
    for _ in range(5):
        print(f.readline(), end='')
    ratio = os.path.getsize(data01) / os.path.getsize(DATAFILE)
    print(f'\n ratio: {ratio:0.2f}')
check_gen_sample_result()
```

```
FECHA_INSCRIPCION;GENERO_DESC;DISTRITO_COD;DISTRITO_DESC;EDAD;NACIONALIDAD_DESC;OBJETIVOPROFESIONAL1_COD;OBJETIVOPROFESIONAL1_DESC;OBJETIVOPROFESIONAL2_COD;OBJETIVOPROFESIONAL2_DESC;OBJETIVOPROFESIONAL3_COD;OBJETIVOPROFESIONAL3_DESC;FX_CARGA
"ago-24";"Mujer";13;" PUENTE DE VALLECAS";51;"Extracomunitario";"5811";"Peluqueros";"9100";"Empleados domésticos";"5220";"Vendedores en tiendas y almacenes";"2025-08-07 00:11:57.897"
"jun-23";"Hombre";11;" CARABANCHEL";25;"Extracomunitario";"7221";"Fontaneros";"7191";"Mantenedores de edificios";"9602";"Peones de la construcción de edificios";"2025-08-07 00:11:57.897"
"jul-17";"Mujer";22;" OTRO MUNICIPIO";45;"Español";"5722";"Cuidadores de niños en domicilios";"9210";"Personal de limpieza de oficinas, hoteles y otros establecimientos similares";"9700";"Peones de las industrias manufactureras";"2025-08-07 00:11:57.897"
"mar-23";"Hombre";19;" VICÁLVARO";32;"Extracomunitario";"5833";"Conserjes de edificios";"7314";"Montadores de estructuras metálicas";"9700";"Peones de las industrias manufactureras";"2025-08-07 00:11:57.897"
```

ratio: 0.01

La instrucción anterior da una salida similar a esta:

```
FECHA_INSCRIPCION;GENERO_DESC;DISTRITO_COD;DISTRITO_DESC;EDAD;NACIONALIDAD_DESC;OBJETIVOPROFESIONAL1_COD;OBJETIVOPROFESIONAL1_DESC;OBJETIVOPROFESIONAL2_COD;OBJETIVOPROFESIONAL2_DESC;OBJETIVOPROFESIONAL3_COD;OBJETIVOPROFESIONAL3_DESC;FX_CARGA
"jun-21";"Hombre";13;" PUENTE DE VALLECAS";36;"Extracomunitario";"8333";"Operadores de carretillas elevadoras";"8412";"Conductores asalarados de automóviles, taxis y furgonetas";"9700";"Peones de las industrias manufactureras";"2025-08-07 00:11:57.897"
"sep-24";"Mujer";14;" MORATALAZ";28;"Español";"9811";"Peones del transporte de mercancías y descargadores";"9811";"Peones del transporte de mercancías y descargadores";"9700";"Peones de las industrias manufactureras";"2025-08-07 00:11:57.897"
"dic-18";"Hombre";10;" LATINA";33;"Comunitario";"3811";"Técnicos en operaciones de sistemas informáticos";"9820";"Reponedores";"9700";"Peones de las industrias manufactureras";"2025-08-07 00:11:57.897"
"oct-22";"Hombre";12;" USERA";60;"Español";"5833";"Conserjes de edificios";"5942";"Auxiliares de vigilante de seguridad y similares no habilitados para ir armados";"9700";"Peones de las industrias manufactureras";"2025-08-07 00:11:57.897"
|
ratio: 0.01
```

Puedes comprobar los doctests de tu función.

```
In [9]: doctest.run_docstring_examples(gen_sample, globals(), verbose=True)
```

Finding tests in NoName

Trying:

```
try:
    gen_sample("dummy.csv", 1.5)
except ValueError as e:
    print(e)
```

Expecting:

La probabilidad debe estar entre 0 y 1.

ok

Trying:

```
base, ext = os.path.splitext("inscritos.csv")
```

Expecting nothing

ok

Trying:

```
p_test = 0.05
```

Expecting nothing

ok

Trying:

```
suffix = f"_{int(p_test * 100):02d}"
```

Expecting nothing

ok

Trying:

```
out_name = f"{base}{suffix}{ext}"
```

Expecting nothing

ok

Trying:

```
out_name
```

Expecting:

```
'inscritos_05.csv'
```

ok

Comprueba la calidad del docstring

```
In [10]: help(gen_sample)
```

Help on function gen_sample in module __main__:

gen_sample(file_name: str, p: float) -> None

Genera un archivo de muestra seleccionando líneas aleatorias del archivo.

Esta función crea subconjuntos de datos más manejables cuando trabajamos con datasets que consumen mucha memoria.

Args:

file_name: archivo csv original

p: probabilidad de selección ($0 < p < 1$)

Returns:

None. Crea un archivo en disco con el nombre modificado de salida generad

o

(datos_05.csv en caso de $p = 0.05$, datos_01 si $p = 0.01$, etc.)

Raises:

Error si p no está en el rango (0,1)

Examples:

Prueba de excepción para porcentaje fuera de rango

```
>>> try:
```

```
...     gen_sample("dummy.csv", 1.5)
```

```
... except ValueError as e:
```

```
...     print(e)
```

La probabilidad debe estar entre 0 y 1.

Prueba del nombre de archivo de salida

```
>>> base, ext = os.path.splitext("inscritos.csv")
```

```
>>> p_test = 0.05
```

```
>>> suffix = f"_{int(p_test * 100):02d}"
```

```
>>> out_name = f"{base}{suffix}{ext}"
```

```
>>> out_name
```

```
'inscritos_05.csv'
```

Comprobación automática de tu código

```
In [11]: testing.test_gen_sample(gen_sample)
```

```
test_gen_sample 0.01 1
```

```
Sample size 0.01 OK
```

```
test_gen_sample 0.02 1
```

```
Sample size 0.02 OK
```

```
test_gen_sample 0.05 1
```

```
Sample size 0.05 OK
```

```
test_gen_sample 0.1 1
```

```
Sample size 0.1 OK
```

```
In [12]: testing.test_gen_sample_typehints(gen_sample)
```

```
check type hints OK
```

```
In [13]: testing.test_docstring(gen_sample)
```

```
docstring OK
```

```
In [14]: testing.test_doctests(gen_sample, globals())
```

doctests OK

Lectura del fichero

Vamos a hacer una función que lea el fichero de datos y lo almacene en una variable para luego usarla para analizar datos.

Para ello vamos a ver cómo es la cabecera y la primera línea del fichero de datos.

```
In [15]: with open(DATAFILE, encoding='utf-8-sig') as f:
        header = f.readline()
        print(header.strip())
        line = f.readline()
        print(line.strip())
```

```
FECHA_INSCRIPCION;GENERO_DESC;DISTRITO_COD;DISTRITO_DESC;EDAD;NACIONALIDAD_DESC;O
BJETIVOPROFESIONAL1_COD;OBJETIVOPROFESIONAL1_DESC;OBJETIVOPROFESIONAL2_COD;OBJETI
VOPROFESIONAL2_DESC;OBJETIVOPROFESIONAL3_COD;OBJETIVOPROFESIONAL3_DESC;FX_CARGA
"ene-24";"Hombre";10;" LATINA";34;"Español";"5833";"Conserjes de edificios";"522
0";"Vendedores en tiendas y almacenes";"9602";"Peones de la construcción de edifi
cios";"2025-08-07 00:11:57.897"
```

Vemos que los datos están separados por puntos y comas `;`, y algunos datos están entre comillas `"`. Por tanto, para leerlo deberemos usar la clase `DictReader` del módulo `csv` de forma adecuada. Al hacer la lectura con el `DictReader`, vamos a obtener un diccionario con el siguiente aspecto:

```
{'DISTRITO_COD': '10',
 'DISTRITO_DESC': ' LATINA',
 'EDAD': '34',
 'FECHA_INSCRIPCION': 'ene-24',
 'FX_CARGA': '2025-08-07 00:11:57.897',
 'GENERO_DESC': 'Hombre',
 'NACIONALIDAD_DESC': 'Español',
 'OBJETIVOPROFESIONAL1_COD': '5833',
 'OBJETIVOPROFESIONAL1_DESC': 'Conserjes de edificios',
 'OBJETIVOPROFESIONAL2_COD': '5220',
 'OBJETIVOPROFESIONAL2_DESC': 'Vendedores en tiendas y
almacenes',
 'OBJETIVOPROFESIONAL3_COD': '9602',
 'OBJETIVOPROFESIONAL3_DESC': 'Peones de la construcción de
edificios'}
```

En este diccionario, todos los valores son cadenas de caracteres. Necesitaremos funciones para transformar los datos que no sean cadenas de caracteres en datos numéricos (`int`) o fechas (`datetime.datetime`). En cualquier caso, no podremos asumir que los datos sean correctos: pueden no estar consignados, o ser incorrectos.

Función `str2int`

Realizaremos una función `str2int` de forma que tome una cadena de caracteres y la transforme en el correspondiente entero. En caso de que el entero no esté o esté de

forma incorrecta, la función debe devolver un 0.

```
In [16]: def str2int(s: str) -> int:
        """
        Convierte una cadena de texto a entero de forma segura, implementando un pat

        Args:
            s: Cadena de texto que representa un nº entero

        Returns:
            Entero convertido, o 0 si la conversión falla

        Examples:
            >>> str2int('5283')
            5283
            >>> str2int('')
            0
            >>> str2int('wrong')
            0
            >>> str2int('71')
            71
        """
        try:
            # Intenta conversión directa a entero
            return int(s)
        except (ValueError, TypeError):
            # ValueError: si s no es numérico
            # TypeError: si s es None u otro tipo incompatible
            return 0
```

```
In [17]: # En esta casilla, puedes generar tus propias pruebas.
```

```
print(str2int('5283'))    # Debe imprimir: 5283
print(str2int(''))        # Debe imprimir: 0
print(str2int('wrong'))   # Debe imprimir: 0
print(str2int('71'))      # Debe imprimir: 71
```

```
5283
0
0
71
```

Autocomprobación

```
In [18]: def check_str2int():
        n = str2int('1234')
        print(f"str2int 1: {n}")

        n = str2int('')
        print(f"str2int 2: {n}")

        n = str2int('wrong')
        print(f"str2int 3: {n}")

        check_str2int()
```

```
str2int 1: 1234
str2int 2: 0
str2int 3: 0
```

Salida esperada:

```
str2int 1: 1234
str2int 2: 0
str2int 3: 0
```

Puedes comprobar los doctests de tu función.

```
In [19]: doctest.run_docstring_examples(str2int, globals(), verbose=True)
```

Finding tests in NoName

Trying:

```
    str2int('5283')
```

Expecting:

```
    5283
```

ok

Trying:

```
    str2int('')
```

Expecting:

```
    0
```

ok

Trying:

```
    str2int('wrong')
```

Expecting:

```
    0
```

ok

Trying:

```
    str2int('71')
```

Expecting:

```
    71
```

ok

Comprueba la calidad del docstring

```
In [20]: help(str2int)
```

Help on function str2int in module __main__:

str2int(s: str) -> int

Convierte una cadena de texto a entero de forma segura, implementando un patrón que retorna 0 para valores inválidos.

Args:

s: Cadena de texto que representa un nº entero

Returns:

Entero convertido, o 0 si la conversión falla

Examples:

```
>>> str2int('5283')
5283
>>> str2int('')
0
>>> str2int('wrong')
0
>>> str2int('71')
71
```

In [21]: `### Comprobaciones automáticas`

```
testing.test_str2int(str2int)
```

str2int OK

In [22]: `testing.test_docstring(str2int)`

docstring OK

In [23]: `testing.test_str2int_typehints(str2int)`

str2int type hints OK

In [24]: `testing.test_doctests(str2int, globals())`

doctests OK

Función str2dt

Vamos a realizar una función que transforme las fechas en cadenas de caracteres como

'2025-08-07 00:11:57.897' del campo `FX_CARGA`, a un objeto de clase `datetime` del módulo `datetime` con los datos correspondientes.

Indicación: usa la función `strptime` de la clase `datetime`

Si el dato es vacío o incorrecto, la función debe devolver dato correspondiente a la fecha 1 de enero de 1970 a las 0 horas.

In [25]: `def str2dt(s: str) -> datetime:`

"""

Convierte una cadena de timestamp a objeto datetime. Formato "YYYY-MM-DD HH:

Args:

s: Cadena con fecha/hora en formato "2025-08-07 00:11:57.897"

Returns:

Objeto datetime, o fecha epoch (1/1/1970) si hay error

Examples:

```
>>> str2dt("2025-08-07 00:11:57.897")
datetime.datetime(2025, 8, 7, 0, 11, 57, 897000)
>>> str2dt("")
datetime.datetime(1970, 1, 1, 0, 0)
>>> str2dt("wrong")
datetime.datetime(1970, 1, 1, 0, 0)
"""
# Convención común para fechas inválidas
default = datetime(1970, 1, 1, 0, 0, 0)

# Manejo de fecha vacía
if not s:
    return default

try:
    #.strptime - string a datetime según formato: "%Y año, %m mes, %d día, %
    return datetime.strptime(s.strip(), "%Y-%m-%d %H:%M:%S.%f")
except (ValueError, TypeError):
    return default
```

Autocomprobación

```
In [26]: def check_str2dt():
d = str2dt("2025-08-07 00:11:57.897")
print(f"str2dt-1: {d}")
d = str2dt("")
print(f"str2dt-2: {d}")
d = str2dt("wrong")
print(f"str2dt-3: {d}")
```

```
check_str2dt()
```

```
str2dt-1: 2025-08-07 00:11:57.897000
```

```
str2dt-2: 1970-01-01 00:00:00
```

```
str2dt-3: 1970-01-01 00:00:00
```

Salida esperada:

```
str2dt-1: 2025-08-07 00:11:57.897000
str2dt-2: 1970-01-01 00:00:00
str2dt-3: 1970-01-01 00:00:00
```

Puedes comprobar los doctests de tu función.

```
In [27]: doctest.run_docstring_examples(str2dt, globals(), verbose=True)
```

```

Finding tests in NoName
Trying:
    str2dt("2025-08-07 00:11:57.897")
Expecting:
    datetime.datetime(2025, 8, 7, 0, 11, 57, 897000)
ok
Trying:
    str2dt("")
Expecting:
    datetime.datetime(1970, 1, 1, 0, 0, 0)
ok
Trying:
    str2dt("wrong")
Expecting:
    datetime.datetime(1970, 1, 1, 0, 0, 0)
ok

```

Comprueba la calidad del docstring

```
In [28]: help(str2dt)
```

Help on function str2dt in module __main__:

```

str2dt(s: str) -> datetime.datetime
    Convierte una cadena de timestamp a objeto datetime. Formato "YYYY-MM-DD HH:M
    M:SS.fff"

```

Args:

s: Cadena con fecha/hora en formato "2025-08-07 00:11:57.897"

Returns:

Objeto datetime, o fecha epoch (1/1/1970) si hay error

Examples:

```

>>> str2dt("2025-08-07 00:11:57.897")
datetime.datetime(2025, 8, 7, 0, 11, 57, 897000)
>>> str2dt("")
datetime.datetime(1970, 1, 1, 0, 0, 0)
>>> str2dt("wrong")
datetime.datetime(1970, 1, 1, 0, 0, 0)

```

Comprobación automática

```
In [29]: testing.test_str2dt(str2dt)
```

```

str2dt OK
str2dt OK

```

```
In [30]: testing.test_str2dt_typehints(str2dt)
```

```
str2dt type hints OK
```

```
In [31]: testing.test_docstring(str2dt)
```

```
docstring OK
```

```
In [32]: testing.test_doctests(str2dt, globals())
```

```
doctests OK
```

Función `str2ym`

En el fichero de datos hay otra cadena de caracteres que se corresponde con una fecha. Es el campo `'FECHA_INSCRIPCION'` y los datos son de la forma `'ene-24'`. En teoría, la función `strptime` se podría usar poniendo la localización de forma adecuada.

Lamentablemente, en la plataforma Windows el siguiente código no da el resultado esperado.

```
In [33]: locale.setlocale(locale.LC_ALL, ("es_ES", "UTF-8"))
print(locale.getlocale())
s = 'ene-24'
try:
    dt = datetime.strptime(s, '%b-%y')
    print(dt)
except ValueError as e:
    print(f"Se ha producido un error al analizar la fecha: {e}")
```

```
('es_ES', 'UTF-8')
```

```
Se ha producido un error al analizar la fecha: time data 'ene-24' does not match
format '%b-%y'
```

Por tanto, no queda más remedio que hacer esta función a mano. Se pide una función llamada `str2ym` que analice un string como el pedido y devuelva la fecha adecuada.

```
In [34]: def str2ym(s: str) -> datetime:
        """
        Convierte una cadena tipo 'ene-24' en un objeto datetime con día=1.

        Args:
            s: cadena en formato 'mes-aa' donde mes son tres letras (en español) rep

        Returns:
            datetime con día 1 del mes/año indicado, o si la cadena es inválida, dev

        Examples:
            >>> str2ym("ene-25")
            datetime.datetime(2025, 1, 1, 0, 0)
            >>> str2ym("dic-24")
            datetime.datetime(2024, 12, 1, 0, 0)
            >>> str2ym("")
            datetime.datetime(1970, 1, 1, 0, 0)
            >>> str2ym("wrong")
            datetime.datetime(1970, 1, 1, 0, 0)
        """
        default = datetime(1970, 1, 1)

        if not s:
            return default

        # Diccionario de mapeo: abreviaturas de meses en español a número de mes
        meses = {
            'ene': 1, 'feb': 2, 'mar': 3, 'abr': 4, 'may': 5, 'jun': 6,
            'jul': 7, 'ago': 8, 'sep': 9, 'oct': 10, 'nov': 11, 'dic': 12
        }
```

```

try:
    # Normalización de minúsculas y limpieza de espacios
    parts = s.strip().lower().split('-')
    if len(parts) != 2:
        return default

    mes_str, anio_str = parts

    # Lookup en diccionario
    mes = meses.get(mes_str)
    if mes is None:
        return default

    # Conversión de año: suponemos 2000 (siglo XXI) + número de dos dígitos
    anio = 2000 + int(anio_str)

    # Devolvemos datetime con el día 1 (principio de mes)
    return datetime(anio, mes, 1)

except (ValueError, TypeError):
    return default

```

Autocomprobación

```

In [35]: def check_str2ym():
    d = str2ym("ene-25")
    print(f"str2ym-1: {d}")
    d = str2ym("")
    print(f"str2ym-2: {d}")
    d = str2ym("wrong")
    print(f"str2ym-3: {d}")

    check_str2ym()

```

```

str2ym-1: 2025-01-01 00:00:00
str2ym-2: 1970-01-01 00:00:00
str2ym-3: 1970-01-01 00:00:00

```

Salida esperada:

```

str2ym-1: 2025-01-01 00:00:00
str2ym-2: 1970-01-01 00:00:00
str2ym-3: 1970-01-01 00:00:00

```

Puedes comprobar los doctests de tu función.

```

In [36]: doctest.run_docstring_examples(str2ym, globals(), verbose=True)

```

```

Finding tests in NoName
Trying:
    str2ym("ene-25")
Expecting:
    datetime.datetime(2025, 1, 1, 0, 0)
ok
Trying:
    str2ym("dic-24")
Expecting:
    datetime.datetime(2024, 12, 1, 0, 0)
ok
Trying:
    str2ym("")
Expecting:
    datetime.datetime(1970, 1, 1, 0, 0)
ok
Trying:
    str2ym("wrong")
Expecting:
    datetime.datetime(1970, 1, 1, 0, 0)
ok

```

Comprueba tú mismo la calidad del docstring

```
In [37]: help(str2ym)
```

Help on function str2ym in module __main__:

```
str2ym(s: str) -> datetime.datetime
    Convierte una cadena tipo 'ene-24' en un objeto datetime con día=1.
```

Args:

s: cadena en formato 'mes-aa' donde mes son tres letras (en español) representativas de cada mes y aa 2 dígitos

Returns:

datetime con día 1 del mes/año indicado, o si la cadena es inválida, devuelve 1970-01-01

Examples:

```

>>> str2ym("ene-25")
datetime.datetime(2025, 1, 1, 0, 0)
>>> str2ym("dic-24")
datetime.datetime(2024, 12, 1, 0, 0)
>>> str2ym("")
datetime.datetime(1970, 1, 1, 0, 0)
>>> str2ym("wrong")
datetime.datetime(1970, 1, 1, 0, 0)

```

Comprobación automática

```
In [38]: testing.test_str2ym(str2ym)
```

Tests str2ym OK

```
In [39]: testing.test_str2ym_typehints(str2ym)
```

str2ym type hints OK

```
In [40]: testing.test_docstring(str2ym)
```

docstring OK

```
In [41]: testing.test_doctests(str2ym, globals())
```

doctests OK

Diccionario de datos

Para almacenar un fichero vamos a usar un diccionario. Las claves serán cada uno de los campos del fichero que estamos tratando. Los valores serán listas de valores, cada campo tiene una lista asociada a cada valor del fichero. Así, los datos de la primera línea estarán en la posición 0 de las correspondientes listas. Los datos de la segunda línea en la posición 1 y así sucesivamente.

Es un diccionario con claves fijas. Así nuestro diccionario se define de la siguiente forma

```
In [42]: class EmploymentData(TypedDict):
    FECHA_INSCRIPCION: list[datetime]
    GENERO_DESC: list[str]
    DISTRITO_COD: list[int]
    DISTRITO_DESC: list[str]
    EDAD: list[int]
    NACIONALIDAD_DESC: list[str]
    OBJETIVOPROFESIONAL1_COD: list[int]
    OBJETIVOPROFESIONAL1_DESC: list[str]
    OBJETIVOPROFESIONAL2_COD: list[int]
    OBJETIVOPROFESIONAL2_DESC: list[str]
    OBJETIVOPROFESIONAL3_COD: list[int]
    OBJETIVOPROFESIONAL3_DESC: list[str]
    FX_CARGA: list[datetime]
```

Un objeto sin datos será:

```
In [43]: emptydata: EmploymentData = \
    {"FECHA_INSCRIPCION": [],
     "GENERO_DESC": [],
     "DISTRITO_COD": [],
     "DISTRITO_DESC": [],
     "EDAD": [],
     "NACIONALIDAD_DESC": [],
     "OBJETIVOPROFESIONAL1_COD": [],
     "OBJETIVOPROFESIONAL1_DESC": [],
     "OBJETIVOPROFESIONAL2_COD": [],
     "OBJETIVOPROFESIONAL2_DESC": [],
     "OBJETIVOPROFESIONAL3_COD": [],
     "OBJETIVOPROFESIONAL3_DESC": [],
     "FX_CARGA": []
    }
```

Función add_row

Es necesario definir una función que tome un diccionario como el que devuelve un `DictReader` y los añade a un diccionario de tipo `EmploymentData`. Recordemos que los diccionarios que devuelve un `DictReader` es un diccionario como el siguiente:

```
{'DISTRITO_COD': '10',
 'DISTRITO_DESC': 'LATINA',
 'EDAD': '34',
 'FECHA_INSCRIPCION': 'ene-24',
 'FX_CARGA': '2025-08-07 00:11:57.897',
 'GENERO_DESC': 'Hombre',
 'NACIONALIDAD_DESC': 'Español',
 'OBJETIVOPROFESIONAL1_COD': '5833',
 'OBJETIVOPROFESIONAL1_DESC': 'Conserjes de edificios',
 'OBJETIVOPROFESIONAL2_COD': '5220',
 'OBJETIVOPROFESIONAL2_DESC': 'Vendedores en tiendas y
almacenes',
 'OBJETIVOPROFESIONAL3_COD': '9602',
 'OBJETIVOPROFESIONAL3_DESC': 'Peones de la construcción de
edificios'}
```

Su tipo es `dict[str, str]`

Realiza una función `add_row(ed: EmploymentData, row: dict[str, str])` que añada `row` en el diccionario `ed`.

```
In [44]: def add_row(data: EmploymentData, row: dict[str, str]) -> None:
        """
        Añade una fila (row) de datos csv a un diccionario EmploymentData (data).

        Transformación de tipos:
        - Strings >> int (códigos, edades)
        - Strings >> datetime (fechas)
        - Strings >> str (se mantienen las descripciones)

        Args:
        data: Diccionario EmploymentData donde añadir los datos
        row: Fila del csv como diccionario str>>str del DictReader

        Examples:
        >>> data = {"FECHA_INSCRIPCION": [], "FX_CARGA": [], "GENERO_DESC": [],
        >>> row = {"FECHA_INSCRIPCION": "ene-24", "FX_CARGA": "2024-02-01 10:00",
        >>> add_row(data, row)
        >>> len(data["GENERO_DESC"])
        1
        """
        data["FECHA_INSCRIPCION"].append(str2ym(row.get("FECHA_INSCRIPCION", "")))
        data["FX_CARGA"].append(str2dt(row.get("FX_CARGA", "")))
        data["GENERO_DESC"].append(row.get("GENERO_DESC", "").strip())
        data["DISTRITO_DESC"].append(row.get("DISTRITO_DESC", "").strip())
        data["NACIONALIDAD_DESC"].append(row.get("NACIONALIDAD_DESC", ""))
        data["OBJETIVOPROFESIONAL1_DESC"].append(row.get("OBJETIVOPROFESIONAL1_DESC", ""))
        data["OBJETIVOPROFESIONAL2_DESC"].append(row.get("OBJETIVOPROFESIONAL2_DESC", ""))
        data["OBJETIVOPROFESIONAL3_DESC"].append(row.get("OBJETIVOPROFESIONAL3_DESC", ""))

        # Códigos numéricos: transformación string>>int
        data["DISTRITO_COD"].append(str2int(row.get("DISTRITO_COD", "")))
```

```
data["EDAD"].append(str2int(row.get("EDAD", "")))
data["OBJETIVOPROFESIONAL1_COD"].append(str2int(row.get("OBJETIVOPROFESIONAL1_COD", "")))
data["OBJETIVOPROFESIONAL2_COD"].append(str2int(row.get("OBJETIVOPROFESIONAL2_COD", "")))
data["OBJETIVOPROFESIONAL3_COD"].append(str2int(row.get("OBJETIVOPROFESIONAL3_COD", "")))
```

Autocomprobación

```
In [45]: def check_add_row():
    data: EmploymentData = \
        {
            "FECHA_INSCRIPCION": [],
            "GENERO_DESC": [],
            "DISTRITO_COD": [],
            "DISTRITO_DESC": [],
            "EDAD": [],
            "NACIONALIDAD_DESC": [],
            "OBJETIVOPROFESIONAL1_COD": [],
            "OBJETIVOPROFESIONAL1_DESC": [],
            "OBJETIVOPROFESIONAL2_COD": [],
            "OBJETIVOPROFESIONAL2_DESC": [],
            "OBJETIVOPROFESIONAL3_COD": [],
            "OBJETIVOPROFESIONAL3_DESC": [],
            "FX_CARGA": []
        }

    row = {
        'DISTRITO_COD': '10',
        'DISTRITO_DESC': 'LATINA',
        'EDAD': '34',
        'FECHA_INSCRIPCION': 'ene-24',
        'FX_CARGA': '2025-08-07 00:11:57.897',
        'GENERO_DESC': 'Hombre',
        'NACIONALIDAD_DESC': 'Español',
        'OBJETIVOPROFESIONAL1_COD': '5833',
        'OBJETIVOPROFESIONAL1_DESC': 'Conserjes de edificios',
        'OBJETIVOPROFESIONAL2_COD': '5220',
        'OBJETIVOPROFESIONAL2_DESC': 'Vendedores en tiendas y almacenes',
        'OBJETIVOPROFESIONAL3_COD': '9602',
        'OBJETIVOPROFESIONAL3_DESC': 'Peones de la construcción de edificios'
    }
    add_row(data, row)
    for k, v in data.items():
        print(f"add_data-2: {k}:{v}")

    row = {
        'DISTRITO_COD': '',
        'DISTRITO_DESC': 'LATINA',
        'EDAD': '',
        'FECHA_INSCRIPCION': '',
        'FX_CARGA': '',
        'GENERO_DESC': '',
        'NACIONALIDAD_DESC': '',
        'OBJETIVOPROFESIONAL1_COD': '',
        'OBJETIVOPROFESIONAL1_DESC': '',
        'OBJETIVOPROFESIONAL2_COD': '',
        'OBJETIVOPROFESIONAL2_DESC': '',
        'OBJETIVOPROFESIONAL3_COD': '',
        'OBJETIVOPROFESIONAL3_DESC': ''
    }
    add_row(data, row)
    for k, v in data.items():
        print(f"add_data-2: {k}:{v}")
```

```
check_add_row()
```

```
add_data-2: FECHA_INSCRIPCION:[datetime.datetime(2024, 1, 1, 0, 0)]
add_data-2: GENERO_DESC:['Hombre']
add_data-2: DISTRITO_COD:[10]
add_data-2: DISTRITO_DESC:['LATINA']
add_data-2: EDAD:[34]
add_data-2: NACIONALIDAD_DESC:['Español']
add_data-2: OBJETIVOPROFESIONAL1_COD:[5833]
add_data-2: OBJETIVOPROFESIONAL1_DESC:['Conserjes de edificios']
add_data-2: OBJETIVOPROFESIONAL2_COD:[5220]
add_data-2: OBJETIVOPROFESIONAL2_DESC:['Vendedores en tiendas y almacenes']
add_data-2: OBJETIVOPROFESIONAL3_COD:[9602]
add_data-2: OBJETIVOPROFESIONAL3_DESC:['Peones de la construcción de edificios']
add_data-2: FX_CARGA:[datetime.datetime(2025, 8, 7, 0, 11, 57, 897000)]
add_data-2: FECHA_INSCRIPCION:[datetime.datetime(2024, 1, 1, 0, 0), datetime.date
time(1970, 1, 1, 0, 0)]
add_data-2: GENERO_DESC:['Hombre', '']
add_data-2: DISTRITO_COD:[10, 0]
add_data-2: DISTRITO_DESC:['LATINA', 'LATINA']
add_data-2: EDAD:[34, 0]
add_data-2: NACIONALIDAD_DESC:['Español', '']
add_data-2: OBJETIVOPROFESIONAL1_COD:[5833, 0]
add_data-2: OBJETIVOPROFESIONAL1_DESC:['Conserjes de edificios', '']
add_data-2: OBJETIVOPROFESIONAL2_COD:[5220, 0]
add_data-2: OBJETIVOPROFESIONAL2_DESC:['Vendedores en tiendas y almacenes', '']
add_data-2: OBJETIVOPROFESIONAL3_COD:[9602, 0]
add_data-2: OBJETIVOPROFESIONAL3_DESC:['Peones de la construcción de edificios',
'']
add_data-2: FX_CARGA:[datetime.datetime(2025, 8, 7, 0, 11, 57, 897000), datetime.
datetime(1970, 1, 1, 0, 0)]
```

Salida esperada:

```
add_data-2: FECHA_INSCRIPCION:[datetime.datetime(2024, 1, 1, 0, 0)]
add_data-2: GENERO_DESC:['Hombre']
add_data-2: DISTRITO_COD:[10]
add_data-2: DISTRITO_DESC:['LATINA']
add_data-2: EDAD:[34]
add_data-2: NACIONALIDAD_DESC:['Español']
add_data-2: OBJETIVOPROFESIONAL1_COD:[5833]
add_data-2: OBJETIVOPROFESIONAL1_DESC:['Conserjes de edificios']
add_data-2: OBJETIVOPROFESIONAL2_COD:[5220]
add_data-2: OBJETIVOPROFESIONAL2_DESC:['Vendedores en tiendas y almacenes']
add_data-2: OBJETIVOPROFESIONAL3_COD:[9602]
add_data-2: OBJETIVOPROFESIONAL3_DESC:['Peones de la construcción de edificios']
add_data-2: FX_CARGA:[datetime.datetime(2025, 8, 7, 0, 11, 57, 897000)]
add_data-2: FECHA_INSCRIPCION:[datetime.datetime(2024, 1, 1, 0, 0), datetime.datetime(1970, 1, 1, 0, 0)]
add_data-2: GENERO_DESC:['Hombre', '']
add_data-2: DISTRITO_COD:[10, 0]
add_data-2: DISTRITO_DESC:['LATINA', 'LATINA']
add_data-2: EDAD:[34, 0]
add_data-2: NACIONALIDAD_DESC:['Español', '']
add_data-2: OBJETIVOPROFESIONAL1_COD:[5833, 0]
add_data-2: OBJETIVOPROFESIONAL1_DESC:['Conserjes de edificios', '']
add_data-2: OBJETIVOPROFESIONAL2_COD:[5220, 0]
add_data-2: OBJETIVOPROFESIONAL2_DESC:['Vendedores en tiendas y almacenes', '']
add_data-2: OBJETIVOPROFESIONAL3_COD:[9602, 0]
add_data-2: OBJETIVOPROFESIONAL3_DESC:['Peones de la construcción de edificios', '']
add_data-2: FX_CARGA:[datetime.datetime(2025, 8, 7, 0, 11, 57, 897000), datetime.datetime(1970, 1, 1, 0, 0)]
```

Puedes comprobar los doctests de tu función.

```
In [46]: doctest.run_docstring_examples(add_row, globals(), verbose=True)
```

Finding tests in NoName

Trying:

```
data = {"FECHA_INSCRIPCION": [], "FX_CARGA": [], "GENERO_DESC": [], "DISTRITO_DESC": [], "NACIONALIDAD_DESC": [], "OBJETIVOPROFESIONAL1_DESC": [], "OBJETIVOPROFESIONAL2_DESC": [], "OBJETIVOPROFESIONAL3_DESC": [], "DISTRITO_COD": [], "EDAD": [], "OBJETIVOPROFESIONAL1_COD": [], "OBJETIVOPROFESIONAL2_COD": [], "OBJETIVOPROFESIONAL3_COD": []}
```

Expecting nothing

ok

Trying:

```
row = {"FECHA_INSCRIPCION": "ene-24", "FX_CARGA": "2024-02-01 10:00:00.000", "GENERO_DESC": "Hombre", "DISTRITO_DESC": "Centro", "NACIONALIDAD_DESC": "Español", "OBJETIVOPROFESIONAL1_DESC": "Ventas", "OBJETIVOPROFESIONAL2_DESC": "", "OBJETIVOPROFESIONAL3_DESC": "", "DISTRITO_COD": "1", "EDAD": "35", "OBJETIVOPROFESIONAL1_COD": "111", "OBJETIVOPROFESIONAL2_COD": "0", "OBJETIVOPROFESIONAL3_COD": "0"}
```

Expecting nothing

ok

Trying:

```
add_row(data, row)
```

Expecting nothing

ok

Trying:

```
len(data["GENERO_DESC"])
```

Expecting:

```
1
```

ok

Comprueba la calidad del docstring

In [47]: `help(add_row)`

Help on function add_row in module __main__:

```
add_row(data: __main__.EmploymentData, row: dict[str, str]) -> None
    Añade una fila (row) de datos csv a un diccionario EmploymentData (data).
```

Transformación de tipos:

- Strings >> int (códigos, edades)
- Strings >> datetime (fechas)
- Strings >> str (se mantienen las descripciones)

Args:

data: Diccionario EmploymentData donde añadir los datos
row: Fila del csv como diccionario str>>str del DictReader

Examples:

```
>>> data = {"FECHA_INSCRIPCION": [], "FX_CARGA": [], "GENERO_DESC": [],
"DISTRITO_DESC": [], "NACIONALIDAD_DESC": [], "OBJETIVOPROFESIONAL1_DESC": [], "O
BJETIVOPROFESIONAL2_DESC": [], "OBJETIVOPROFESIONAL3_DESC": [], "DISTRITO_COD":
[], "EDAD": [], "OBJETIVOPROFESIONAL1_COD": [], "OBJETIVOPROFESIONAL2_COD": [],
"OBJETIVOPROFESIONAL3_COD": []}
>>> row = {"FECHA_INSCRIPCION": "ene-24", "FX_CARGA": "2024-02-01 10:00:0
0.000", "GENERO_DESC": "Hombre", "DISTRITO_DESC": "Centro", "NACIONALIDAD_DESC":
"Español", "OBJETIVOPROFESIONAL1_DESC": "Ventas", "OBJETIVOPROFESIONAL2_DESC":
"", "OBJETIVOPROFESIONAL3_DESC": "", "DISTRITO_COD": "1", "EDAD": "35", "OBJETIVO
PROFESIONAL1_COD": "111", "OBJETIVOPROFESIONAL2_COD": "0", "OBJETIVOPROFESIONAL3_
COD": "0"}
>>> add_row(data, row)
>>> len(data["GENERO_DESC"])
1
```

Comprobación automática

```
In [48]: testing.test_add_row(add_row)
```

add_row OK
add_row with missing values OK

```
In [49]: testing.test_add_row_typehints(add_row)
```

add_row type hints OK

```
In [50]: testing.test_docstring(add_row)
```

docstring OK

```
In [51]: testing.test_doctests(add_row, globals())
```

doctests OK

Función load_employment

Debes realizar una función que lea un fichero de datos de empleo y devuelva un diccionario `EmploymentData`.

```
In [52]: def load_employment(filename: str) -> EmploymentData:
```

```
    """
```

```
    Carga un archivo csv de datos de empleo y devuelve un diccionario Employment
```

```

- Extrae: Lee el csv con DictReader (maneja delimitadores y quotes)
- Transforma: Convierte tipos con add_row()
- Carga: Almacena en un diccionario de listas

Args:
    filename: Ruta del archivo csv con datos de empleo

Returns:
    EmploymentData con todos los registros cargados

Examples:
    >>> data = load_employment("inscritos_test_sample.csv")
    >>> len(data["EDAD"]) > 0
    True
    """
# Creamos un diccionario vacío con estructura de columnas
data: EmploymentData = {
    "FECHA_INSCRIPCION": [],
    "GENERO_DESC": [],
    "DISTRITO_COD": [],
    "DISTRITO_DESC": [],
    "EDAD": [],
    "NACIONALIDAD_DESC": [],
    "OBJETIVOPROFESIONAL1_COD": [],
    "OBJETIVOPROFESIONAL1_DESC": [],
    "OBJETIVOPROFESIONAL2_COD": [],
    "OBJETIVOPROFESIONAL2_DESC": [],
    "OBJETIVOPROFESIONAL3_COD": [],
    "OBJETIVOPROFESIONAL3_DESC": [],
    "FX_CARGA": []
}

with open(filename, encoding="utf-8-sig") as f:
    # DictReader: convierte cada fila en dict usando la primera fila como key
    # delimiter=";": indica ";" como separador
    # quotechar='\"': maneja campos con texto que contiene separadores
    reader = csv.DictReader(f, delimiter=";", quotechar='\"')
    for row in reader:
        add_row(data, row)

return data

```

Autocomprobación

```

In [53]: def check_load_employment():
    ed = load_employment(TESTDATAFILE)
    # ed = load_employment(DATAFILE)
    print(f"load_employment-1: {len(ed['FECHA_INSCRIPCION'])}")
    print(f"load_employment-2: {len(ed['EDAD'])}")
    print(f"load_employment-3: {ed['EDAD'][7]}")
    print(f"load_employment-4: {len([d for d in ed['DISTRITO_COD'] if d==1])}")
    print(f"load_employment-5: {len([d for d in ed['FECHA_INSCRIPCION'] if d.mon

check_load_employment()

```

```
load_employment-1: 1508
load_employment-2: 1508
load_employment-3: 50
load_employment-4: 48
load_employment-5: 100
```

Salida esperada:

```
load_employment-1: 1508
load_employment-2: 1508
load_employment-3: 50
load_employment-4: 48
load_employment-5: 100
```

Comprueba la calidad del docstring

```
In [54]: help(load_employment)
```

Help on function load_employment in module __main__:

load_employment(filename: str) -> __main__.EmploymentData

Carga un archivo csv de datos de empleo y devuelve un diccionario EmploymentData.

- Extrae: Lee el csv con DictReader (maneja delimitadores y quotes)
- Transforma: Convierte tipos con add_row()
- Carga: Almacena en un diccionario de listas

Args:

filename: Ruta del archivo csv con datos de empleo

Returns:

EmploymentData con todos los registros cargados

Examples:

```
>>> data = load_employment("inscritos_test_sample.csv")
>>> len(data["EDAD"]) > 0
True
```

Puedes comprobar los doctests de tu función.

```
In [55]: doctest.run_docstring_examples(load_employment, globals(), verbose=True)
```

Finding tests in NoName

Trying:

```
data = load_employment("inscritos_test_sample.csv")
```

Expecting nothing

ok

Trying:

```
len(data["EDAD"]) > 0
```

Expecting:

```
True
```

ok

Comprobación automática

```
In [56]: testing.test_load_employment(load_employment)
```

load_employment OK

```
In [57]: testing.test_load_employment_typehints(load_employment)
```

```
test: {"<class ' __main__.EmploymentData'>", "<class 'str'>"}
load_employment type hints OK
```

```
In [58]: testing.test_docstring(load_employment)
```

```
docstring OK
```

```
In [59]: testing.test_doctests(load_employment, globals())
```

```
doctests OK
```

Extracción de información

En este apartado queremos sacar cierta información sobre los registros de empleo.

Para ello vamos a guardar en una variable todos los registros de empleo

```
In [60]: global employment_data
employment_data = load_employment('inscritos.csv')
employment_test_data = load_employment('inscritos_test_sample.csv')
```

Función people_by_district

Esta función debe devolver una lista ordenada de tuplas. El primer elemento de cada tupa es el nombre del distrito y el segundo es el número de personas inscritas en el distrito. La lista está ordenada de forma descendente con respecto al número de personas inscritas.

```
In [61]: def people_by_district(ed: EmploymentData) -> list[tuple[str, int]]:
        """
        Cuenta y ordena el nº de personas inscritas por distrito (similar a GROUP BY

        Args:
            ed: Datos de empleo en formato EmploymentData

        Returns:
            Lista de tuplas (nombre_distrito, num_personas) ordenada descendente

        Examples
            >>> data = load_employment("inscritos_test_sample.csv")
            >>> result = people_by_district(data)
            >>> result[0][1] >= result[1][1] # Primer distrito tiene más o igual pe
            True
        """

        # Contamos cuántas veces aparece cada distrito
        counter = Counter(ed["DISTRITO_DESC"])

        # Convertimos el contador en lista de tuplas y ordenamos
        # key=lambda x: x[1] --> ordena por el segundo elemento (count)
        # reverse=True --> ordena de manera descendente (mayor a menor)
        return sorted(counter.items(), key=lambda x: x[1], reverse=True)
```

```
In [62]: # Casilla para probar
```

```
data = employment_test_data

result = people_by_district(data)

# El primer distrito tiene más o igual personas que el segundo
print(result[0][1] >= result[1][1]) # Debe imprimir: True
```

True

Autocomprobación

```
In [63]: def check_people_by_district():
          global employment_data

          for district, people in people_by_district(employment_data):
              print(f'{district:.<25s}{{people:>6d}}')

          check_people_by_district()
```

```
OTRO MUNICIPIO..... 23768
PUENTE DE VALLECAS..... 15356
CARABANCHEL..... 13022
LATINA..... 10251
VILLAVERDE..... 9995
CIUDAD LINEAL..... 7897
USERA..... 7560
TETUÁN..... 7433
SAN BLAS - CANILLEJAS.... 6059
VICÁLVARO..... 5678
VILLA DE VALLECAS..... 5510
ARGANZUELA..... 5460
CENTRO..... 5094
FUENCARRAL-EL PARDO..... 4691
HORTALEZA..... 4534
MORATALAZ..... 3754
RETIRO..... 2770
CHAMBERÍ..... 2498
CHAMARTÍN..... 2255
BARAJAS..... 2255
SALAMANCA..... 2247
MONCLOA-ARAVACA..... 2050
```

Salida esperada:

```
OTRO MUNICIPIO..... 23769
PUENTE DE VALLECAS..... 15357
CARABANCHEL..... 13023
LATINA..... 10252
VILLAVERDE..... 9996
CIUDAD LINEAL..... 7898
USERA..... 7561
TETUÁN..... 7434
SAN BLAS - CANILLEJAS.... 6060
VICÁLVARO..... 5679
VILLA DE VALLECAS..... 5511
ARGANZUELA..... 5461
CENTRO..... 5095
FUENCARRAL-EL PARDO..... 4692
HORTALEZA..... 4535
MORATALAZ..... 3755
RETIRO..... 2771
CHAMBERÍ..... 2499
CHAMARTÍN..... 2256
BARAJAS..... 2256
SALAMANCA..... 2248
MONCLOA-ARAVACA..... 2051
```

Puedes comprobar los doctests de tu función.

```
In [64]: doctest.run_docstring_examples(people_by_district, globals(), verbose=True)
```

```
Finding tests in NoName
Trying:
    data = load_employment("inscritos_test_sample.csv")
Expecting nothing
ok
Trying:
    result = people_by_district(data)
Expecting nothing
ok
Trying:
    result[0][1] >= result[1][1] # Primer distrito tiene más o igual personas
Expecting:
    True
ok
```

Comprueba la calidad del docstring

```
In [65]: help(people_by_district)
```

Help on function people_by_district in module __main__:

people_by_district(ed: __main__.EmploymentData) -> list[tuple[str, int]]

Cuenta y ordena el nº de personas inscritas por distrito (similar a GROUP BY + ORDER BY en SQL)

Args:

ed: Datos de empleo en formato EmploymentData

Returns:

Lista de tuplas (nombre_distrito, num_personas) ordenada descendientemente

Examples

```
>>> data = load_employment("inscritos_test_sample.csv")
```

```
>>> result = people_by_district(data)
```

```
>>> result[0][1] >= result[1][1] # Primer distrito tiene más o igual per
sonas
```

```
True
```

Comprobación automática

```
In [66]: data = load_employment('inscritos_test_sample.csv')
         people_by_district(data)
```

```
Out[66]: [('OTRO MUNICIPIO', 268),
          ('PUENTE DE VALLECAS', 141),
          ('CARABANCHEL', 124),
          ('LATINA', 123),
          ('VILLAVERDE', 94),
          ('CIUDAD LINEAL', 90),
          ('VILLA DE VALLECAS', 73),
          ('USERA', 71),
          ('TETUÁN', 71),
          ('VICÁLVARO', 54),
          ('FUENCARRAL-EL PARDO', 50),
          ('CENTRO', 48),
          ('ARGANZUELA', 46),
          ('HORTALEZA', 45),
          ('MORATALAZ', 41),
          ('SAN BLAS - CANILLEJAS', 41),
          ('CHAMBERÍ', 26),
          ('SALAMANCA', 25),
          ('RETIRO', 23),
          ('CHAMARTÍN', 21),
          ('MONCLOA-ARAVACA', 18),
          ('BARAJAS', 15)]
```

```
In [67]: testing.test_people_by_district(people_by_district)
```

people_by_district OK

```
In [68]: testing.test_people_by_district_typehints(people_by_district)
```

people_by_district type hints OK

```
In [69]: testing.test_doctests(people_by_district, globals())
```

doctests OK

```
In [70]: testing.test_docstring(people_by_district)
```

docstring OK

Función mean_age_by_district

Realiza una función que calcule la edad media de cada distrito. Debe devolver un diccionario, la clave es el nombre del distrito y el valor la edad media.

NOTA Si hay alguna fila que no tiene la edad, lo hemos indicado como que su edad es 0, no debe ser computado a la hora de calcular la media. Es como si ese dato no existiera.

```
In [71]: def mean_age_by_district(ed: EmploymentData) -> dict[str, float]:
          """
          Calcula la edad media por distrito, excluyendo valores no válidos (edad=0)

          Args:
              ed: Datos de empleo en formato EmploymentData

          Returns:
              Diccionario {nombre_distrito: edad_media}

          Examples:
```

```

>>> data = load_employment("inscritos_test_sample.csv")
>>> means = mean_age_by_district(data)
>>> all(age > 0 for age in means.values())
True
"""

# Diccionarios para acumular sumas y conteos (alternativa a if-else)
# defaultdict: evita KeyError, inicia con int() = 0
age_sum = defaultdict(int) # Acumula la suma de edades
age_count = defaultdict(int) # Cuenta los registros válidos

# Iteración paralela sobre dos columnas
# Equivalencia en SQL: SELECT distrito, AVG(edad) GROUP BY distrito WHERE edad > 0
for edad, distrito in zip(ed["EDAD"], ed["DISTRITO_DESC"]):
    if edad > 0: # ignoramos edades 0 (valores no válidos)
        age_sum[distrito] += edad
        age_count[distrito] += 1

# Calculamos media por distrito
# age_count[distrito] > 0 evita división entre 0
mean_age = {distrito: age_sum[distrito] / age_count[distrito]
             for distrito in age_sum if age_count[distrito] > 0}

return mean_age

```

```

In [72]: # Espacio para pruebas

data = employment_test_data

means = mean_age_by_district(data)

# Todas las edades deben ser mayores que 0
print(all(age > 0 for age in means.values())) # Debe imprimir: True

```

True

Autocomprobación

```

In [73]: def check_mean_age_by_district():
          global employment_data
          for district, mage in mean_age_by_district(employment_test_data).items():
              print(f'{district:.<25s}{mage:>6.2f}')
          check_mean_age_by_district()

```

```

OTRO MUNICIPIO..... 37.83
MORATALAZ..... 38.20
PUENTE DE VALLECAS..... 40.36
USERA..... 38.51
CIUDAD LINEAL..... 40.07
CARABANCHEL..... 38.39
HORTALEZA..... 41.00
VILLASVERDE..... 40.77
LATINA..... 42.11
ARGANZUELA..... 38.74
VILLA DE VALLECAS..... 42.77
CENTRO..... 39.96
MONCLOA-ARAVACA..... 42.78
RETIRO..... 38.26
VICÁLVARO..... 38.30
FUENCARRAL-EL PARDO..... 39.30
CHAMBERÍ..... 46.00
CHAMARTÍN..... 40.33
BARAJAS..... 42.93
SAN BLAS - CANILLEJAS.... 36.46
TETUÁN..... 37.48
SALAMANCA..... 43.44

```

Salida esperada:

```

OTRO MUNICIPIO..... 37.83
MORATALAZ..... 38.20
PUENTE DE VALLECAS..... 40.36
USERA..... 38.51
CIUDAD LINEAL..... 40.07
CARABANCHEL..... 38.39
HORTALEZA..... 41.00
VILLASVERDE..... 40.77
LATINA..... 42.11
ARGANZUELA..... 38.74
VILLA DE VALLECAS..... 42.77
CENTRO..... 39.96
MONCLOA-ARAVACA..... 42.78
RETIRO..... 38.26
VICÁLVARO..... 38.30
FUENCARRAL-EL PARDO..... 39.30
CHAMBERÍ..... 46.00
CHAMARTÍN..... 40.33
BARAJAS..... 42.93
SAN BLAS - CANILLEJAS.... 36.46
TETUÁN..... 37.48
SALAMANCA..... 43.44

```

Comprueba la calidad del docstring

```
In [74]: help(mean_age_by_district)
```

Help on function mean_age_by_district in module __main__:

```
mean_age_by_district(ed: __main__.EmploymentData) -> dict[str, float]
    Calcula la edad media por distrito, excluyendo valores no válidos (edad=0)
```

Args:

ed: Datos de empleo en formato EmploymentData

Returns:

Diccionario {nombre_distrito: edad_media}

Examples:

```
>>> data = load_employment("inscritos_test_sample.csv")
>>> means = mean_age_by_district(data)
>>> all(age > 0 for age in means.values())
True
```

Puedes comprobar los doctests de tu función.

```
In [75]: doctest.run_docstring_examples(mean_age_by_district, globals(), verbose=True)
```

Finding tests in NoName

Trying:

```
data = load_employment("inscritos_test_sample.csv")
```

Expecting nothing

ok

Trying:

```
means = mean_age_by_district(data)
```

Expecting nothing

ok

Trying:

```
all(age > 0 for age in means.values())
```

Expecting:

```
True
```

ok

Comprueba la calidad del docstring

Comprobación automática

```
In [76]: testing.test_mean_age_by_district(mean_age_by_district)
```

mean_age_by_district OK

```
In [77]: testing.test_mean_age_by_district(mean_age_by_district)
```

mean_age_by_district OK

```
In [78]: testing.test_docstring(mean_age_by_district)
```

docstring OK

```
In [79]: testing.test_doctests(mean_age_by_district, globals())
```

doctests OK

Ejercicios con numpy

En los ejercicios siguientes se practicará la librería `numpy`, que se importará con el nombre `np` como de costumbre.

Función `year_month_data`

A continuación construiremos a partir de un diccionario de tipo `EmploymentData`, un array de `numpy` de forma que haya una fila por cada año y una columna por cada mes. En cada casilla habrá el número de solicitudes del año y mes correspondiente.

La fila 0 se corresponderá al mínimo de los años que aparezca, y habrá una fila por cada uno de los años hasta el máximo de los años posibles. Si en algún año no hay inscripciones deberá aparecer una fila rellena con ceros. El mes 0 se corresponde con enero y el mes 11 con diciembre.

```
In [80]: def year_month_data(ed: EmploymentData) -> 'tuple[numpy.array, int]':
        """
        Crea una matriz NumPy de inscripciones con filas=años, columnas=meses

        Args:
            ed: Datos de empleo (EmploymentData)

        Returns:
            Tupla (matriz, anio_inicial) donde:
            - La matriz de Numpy (filas=años, columnas=meses)
            - Año inicial correspondiente a la primera fila

        Examples:
            >>> data = load_employment("inscritos_test_sample.csv")
            >>> arr, year = year_month_data(data)
            >>> arr.shape[1] == 12    # 12 meses
            True
        """
        fechas = ed["FECHA_INSCRIPCION"]
        if not fechas:
            return np.zeros((0, 12), dtype=int), None

        anios = np.array([d.year for d in fechas])
        meses = np.array([d.month for d in fechas])

        min_year = anios.min()
        max_year = anios.max()

        # Histogram2d para contar pares de datos
        # Definimos los bordes de los bins para que caigan los años y meses exactos
        bins_x = np.arange(min_year, max_year + 2)
        bins_y = np.arange(1, 14) # Meses 1 a 12

        # Hacemos el histograma
        matriz, _, _ = np.histogram2d(anios, meses, bins=[bins_x, bins_y])

        return matriz.astype(int), min_year
```

```
In [81]: # Casilla para tus pruebas
```

```

data = employment_test_data

arr, year = year_month_data(data)

# La matriz debe tener 12 columnas
print(arr.shape[1] == 12) # Debe imprimir: True

print("Matriz resultante:\n", arr)
print("Año inicial:", year)

```

True

Matriz resultante:

```

[[ 9 10  9 10  3 20 17 10  7  7 24 15]
 [13 16 15  8 10  9  6  6  8 13 21 11]
 [12 17 15 11 11 13 15  4 12 24 14 17]
 [14 18  4  7  9 19  8 10 14 17 22 24]
 [ 7 17 19 16 16 19  8  8 14 12 11 14]
 [11 21 13 16 19 24 12 19 25 19 20 18]
 [17 11 21 13 19 13 16 12 10 16 15 17]
 [10 16 22 24 28 17 21  9 10 22 16 13]
 [16  9 11 14 13 61  7  3  0  0  0  0]]

```

Año inicial: 2017

Autocomprobación

```

In [82]: def check_month_data():
          global employment_data
          ym_data, ini_y = year_month_data(employment_data)
          print(f"Año inicial: {ini_y}")
          print(ym_data)

          check_month_data()

```

Año inicial: 2017

```

[[1177 1508 1632 1204 1256 1434 1143  915 1233 1712 1753  978]
 [1258 1365 1398 1321 1371 1155  947  749 1124 1340 1437 1418]
 [1259 1651 1804 1537 1336 1154 1076  784 1341 1911 1586 1290]
 [1185 1511  827  769  822 1370  880  814 1586 2011 1716 1619]
 [ 861 1857 1580 1982 1799 1638  902  727 1326 1121  956 1396]
 [ 804 2129 2257 1677 2014 1727 1338 1511 1711 1939 2292 1800]
 [1665 1699 1860 1219 1606 1448 1469  880 1166 1557 1807 1417]
 [1479 1982 2034 2330 2363 1989 1630  847 1065 1753 1353  861]
 [ 796 1084 1240 1397 1448 5742  828  112  0  0  0  0]]

```

Salida esperada:

```

Año inicial: 2017
[[1177 1508 1632 1204 1256 1434 1143  915 1233 1712 1753  978]
 [1258 1365 1398 1321 1371 1155  947  749 1124 1340 1437 1418]
 [1259 1651 1804 1537 1336 1154 1076  784 1341 1911 1586 1290]
 [1185 1511  827  769  822 1370  880  814 1586 2011 1716 1619]
 [ 861 1857 1580 1982 1799 1638  902  727 1326 1121  956 1396]
 [ 804 2129 2257 1677 2014 1727 1338 1511 1711 1939 2292 1800]
 [1665 1699 1860 1219 1606 1448 1469  880 1166 1557 1807 1417]
 [1479 1982 2034 2330 2363 1989 1630  847 1065 1753 1353  861]
 [ 796 1084 1240 1397 1448 5742  828  112  0  0  0  0]]

```

Comprueba tus doctests

```
In [83]: doctest.run_docstring_examples(year_month_data, globals(), verbose=True)
```

```
Finding tests in NoName
Trying:
    data = load_employment("inscritos_test_sample.csv")
Expecting nothing
ok
Trying:
    arr, year = year_month_data(data)
Expecting nothing
ok
Trying:
    arr.shape[1] == 12    # 12 meses
Expecting:
    True
ok
```

Comprueba tu docstring

```
In [84]: help(year_month_data)
```

Help on function year_month_data in module __main__:

```
year_month_data(ed: __main__.EmploymentData) -> 'tuple[numpy.array, int]'
    Crea una matriz NumPy de inscripciones con filas=años, columnas=meses
```

Args:

```
ed: Datos de empleo (EmploymentData)
```

Returns:

```
Tupla (matriz, anio_inicial) donde:
    - La matriz de Numpy (filas=años, columnas=meses)
    - Año inicial correspondiente a la primera fila
```

Examples:

```
>>> data = load_employment("inscritos_test_sample.csv")
>>> arr, year = year_month_data(data)
>>> arr.shape[1] == 12    # 12 meses
True
```

Corrección automática

```
In [85]: testing.test_year_month_data(year_month_data)
```

year_month_data OK

```
In [86]: testing.test_year_month_data_typehints(year_month_data)
```

year_month_data type hints OK

```
In [87]: testing.test_docstring(year_month_data)
```

docstring OK

```
In [88]: testing.test_doctests(year_month_data, globals())
```

doctests OK

Puedes comprobar los doctests de tu función. Si son correctos no debería salir nada aquí.

Función `sum_by_month_year`

Importante: En esta función y en el resto de ejercicios de `numpy` **NO** se deben usar un bucles (`while`, `for`), solamente funciones de `numpy`.

En esta función se parte de una matriz como la obtenida en el ejercicio anterior. La función devuelve 2 arrays de `numpy`: el primero es la suma de los datos por meses (un array con 12 posiciones); el segundo es la suma por años deberá haber un elemento por cada año.

```
In [89]: def sum_by_month_year(data: np.array) -> 'tuple[numpy.array, numpy.array]':
        """
        Calcula el total de inscripciones por mes y por año.
        - axis=0 suma columnas (verticalmente) >> total por mes
        - axis=1 suma filas (horizontalmente) >> total por año

        Args:
            data: Matriz de datos (años x meses)

        Returns:
            Tupla (by_month, by_year) donde:
            - by_month: suma vertical
            - by_year: suma horizontal

        Examples:
            >>> data = np.array([[1, 2, 3], [4, 5, 6]])
            >>> by_month, by_year = sum_by_month_year(data)
            >>> by_month.tolist()
            [5, 7, 9]
            >>> by_year.tolist()
            [6, 15]
        """

        # Sumamos por columnas (axis=0) para tener el total de cada mes
        # e.g. columna 0 (enero) = suma de todos los eneros
        by_month = np.sum(data, axis=0)

        # Suma por filas (axis=1) para tener el total de cada mes
        # e.g. fila 0 (primer año) = suma de los 12 meses de ese año
        by_year = np.sum(data, axis=1)

        return by_month, by_year
```

```
In [90]: # Casilla para pruebas

data = np.array([[1, 2, 3], [4, 5, 6]])

by_month, by_year = sum_by_month_year(data)

print(by_month.tolist()) # Debe imprimir: [5, 7, 9]
print(by_year.tolist())  # Debe imprimir: [6, 15]
```

```
[5, 7, 9]
[6, 15]
```

Autocomprobación

```
In [91]: def check_sum_by_month_year():
    ym_data2 = \
        np.array([[ 9, 10,  9, 10,  3, 20, 17, 10,  7,  7, 23, 15],\
                  [12, 17, 15, 11, 11, 13, 15,  4, 12, 24, 14, 17],\
                  [17, 11, 21, 13, 19, 13, 16, 12, 10, 16, 15, 17]])
    by_month, by_year = sum_by_month_year(ym_data2)
    print(f"Por meses-1: {by_month}")
    print(f"Por años-1: {by_year}")

    ym_data, ini_y = year_month_data(employment_data)
    by_month, by_year = sum_by_month_year(ym_data)
    print(f"Por meses-1: {by_month}")
    print(f"Por años-1: {by_year}")

    check_sum_by_month_year()
```

```
Por meses-1: [38 38 45 34 33 46 48 26 29 47 52 49]
Por años-1: [140 165 180]
Por meses-1: [10484 14786 14632 13436 14015 17657 10213  7339 10552 13344 12900 1
0779]
Por años-1: [15945 14883 16729 15110 16145 21199 17793 19686 12647]
```

Salida esperada:

```
Por meses-1: [38 38 45 34 33 46 48 26 29 47 52 49]
Por años-1: [140 165 180]
Por meses-1: [10484 14786 14632 13436 14015 17657 10213  7339 10552 13344 12900 10779]
Por años-1: [15945 14883 16729 15110 16145 21199 17793 19686 12647]
```

Comprobación de doctests

```
In [92]: doctest.run_docstring_examples(sum_by_month_year, globals(), verbose=True)
```

```
Finding tests in NoName
Trying:
    data = np.array([[1, 2, 3], [4, 5, 6]])
Expecting nothing
ok
Trying:
    by_month, by_year = sum_by_month_year(data)
Expecting nothing
ok
Trying:
    by_month.tolist()
Expecting:
    [5, 7, 9]
ok
Trying:
    by_year.tolist()
Expecting:
    [6, 15]
ok
```

Comprobación de docstring

In [93]: `help(sum_by_month_year)`

Help on function sum_by_month_year in module __main__:

`sum_by_month_year(data: <built-in function array>) -> 'tuple[numpy.array, numpy.array]'`

Calcula el total de inscripciones por mes y por año.

- axis=0 suma columnas (verticalmente) >> total por mes
- axis=1 suma filas (horizontalmente) >> total por año

Args:

data: Matriz de datos (años x meses)

Returns:

Tupla (by_month, by_year) donde:

- by_month: suma vertical
- by_year: suma horizontal

Examples:

```
>>> data = np.array([[1, 2, 3], [4, 5, 6]])
>>> by_month, by_year = sum_by_month_year(data)
>>> by_month.tolist()
[5, 7, 9]
>>> by_year.tolist()
[6, 15]
```

Comprobaciones automáticas

In [94]: `testing.test_sum_by_month_year(sum_by_month_year)`

```
data: [[ 9 10  9 10  3 20 17 10  7  7 23 15]
 [12 17 15 11 11 13 15  4 12 24 14 17]
 [17 11 21 13 19 13 16 12 10 16 15 17]]
sum_by_month_year OK
```

In [95]: `testing.test_sum_by_month_year_typehints(sum_by_month_year)`

sum_by_month_year type hints OK

In [96]: `testing.test_doctests(sum_by_month_year, globals())`

doctests OK

In [97]: `testing.test_docstring(sum_by_month_year)`

docstring OK

Función `max_requests`

Esta función debe tomar un array de `numpy` como en los casos anteriores y debe devolver el año, mes y valor máximo entre todas las entradas del array. Para saber el año debemos saber qué año se corresponde con la fila 0. Será un parámetro de la función. La función devolverá los valores por este orden: año, mes, valor; y como tipos básicos de python: `int`, `str` e `int` respectivamente.

```
In [98]: def max_requests(data: np.array, ini_year: int) -> 'tuple[int, str, int]':
        """
        Busca el valor máximo de inscripciones y devuelve cuando ocurrió (mes y año)

        Args:
            data: Matriz años x meses
            ini_year: Año correspondiente a la primera fila

        Returns:
            Tupla (anio, mes, valor_maximo) donde:

        Examples:
            >>> data = np.array([[10, 20, 15], [25, 30, 12]])
            >>> year, month, val = max_requests(data, 2020)
            >>> val
            30
            >>> year
            2021
        """

        # Índice 0=Enero, 11=Diciembre
        meses = ['ene', 'feb', 'mar', 'abr', 'may', 'jun', 'jul', 'ago', 'sep', 'oct', 'nov', 'dic']

        # Obtenemos el valor máximo
        max_val = np.max(data)

        # Encontramos dónde está ese valor (filas y columnas)
        # np.where devuelve tupla de arrays: (array_filas, array_columnas)
        posiciones = np.where(data == max_val)

        # Cogemos la primera ocurrencia (por si hay empates)
        # posiciones[0] son filas y posiciones[0] columnas
        fila = posiciones[0][0]
        columna = posiciones[1][0]

        year = ini_year + fila
        month_name = meses[columna]

        return int(year), month_name, int(max_val)
```

```
In [99]: # Casilla para pruebas

data = np.array([[10, 20, 15],
                  [25, 30, 12]])

year, month, val = max_requests(data, 2020)
```

```
print(val)    # Debe imprimir: 30
print(year)   # Debe imprimir: 2021
```

```
30
2021
```

Autocomprobación

In [100...

```
def check_max_requests():
    global employment_data

    ym_data2 = \
        np.array([[ 9, 10,  9, 10,  3, 20, 17, 10,  7,  7, 23, 15],\
                  [12, 17, 15, 11, 11, 13, 15,  4, 12, 24, 14, 17],\
                  [17, 11, 21, 13, 19, 13, 16, 12, 10, 16, 15, 17]])

    year, month, val = max_requests(ym_data2, 2020)
    print(f"Año-1: {year}")
    print(f"Mes-1: {month}")
    print(f"Valor-1: {val}")
    print("\n===== \n")
    ym_data, ini_y = year_month_data(employment_data)
    year, month, val = max_requests(ym_data, 2017)
    print(f"Año-2: {year}")
    print(f"Mes-2: {month}")
    print(f"Valor-2: {val}")

check_max_requests()
```

```
Año-1: 2021
Mes-1: oct
Valor-1: 24
```

```
=====
```

```
Año-2: 2025
Mes-2: jun
Valor-2: 5742
```

Salida esperada:

```
Año-1: 2021
Mes-1: oct
Valor-1: 24
```

```
=====
```

```
Año-2: 2025
Mes-2: jun
Valor-2: 5742
```

Comprobacion de docstring

In [101...

```
help(max_requests)
```

Help on function max_requests in module __main__:

```
max_requests(data: <built-in function array>, ini_year: int) -> 'tuple[int, str, int]'
```

Busca el valor máximo de inscripciones y devuelve cuando ocurrió (mes y año)

Args:

data: Matriz años x meses

ini_year: Año correspondiente a la primera fila

Returns:

Tupla (anio, mes, valor_maximo) donde:

Examples:

```
>>> data = np.array([[10, 20, 15], [25, 30, 12]])
```

```
>>> year, month, val = max_requests(data, 2020)
```

```
>>> val
```

```
30
```

```
>>> year
```

```
2021
```

Comprobación de doctests

```
In [102... doctest.run_docstring_examples(max_requests, globals(), verbose=True)
```

Finding tests in NoName

Trying:

```
data = np.array([[10, 20, 15], [25, 30, 12]])
```

Expecting nothing

ok

Trying:

```
year, month, val = max_requests(data, 2020)
```

Expecting nothing

ok

Trying:

```
val
```

Expecting:

```
30
```

ok

Trying:

```
year
```

Expecting:

```
2021
```

ok

Comprobación automática

```
In [103... testing.test_max_requests(max_requests)
```

```
data: [[ 9 10  9 10  3 20 17 10  7  7 23 15]
```

```
[12 17 15 11 11 13 15  4 12 24 14 17]
```

```
[17 11 21 13 19 13 16 12 10 16 15 17]]
```

max_requests OK

```
In [104... testing.test_max_requests_typehints(max_requests)
```

max_requests type hints OK

```
In [105... testing.test_docstring(max_requests)
```

docstring OK

In [106... `testing.test_doctests(max_requests, globals())`

doctests OK

función `by_quarters`

La función `by_quarters`, a partir de una matriz como en los casos anteriores, devuelve una matriz similar pero con los datos por trimestres. Es decir la matriz tendrá cuatro filas, una por trimestre.

```
In [107... def by_quarters(data: np.array) -> np.array:
    """
    Agrupa los datos mensuales en trimestres: 12 meses -> 4 trimestres
        - Q1: Enero-Marzo (columnas 0-2)
        - Q2: Abril-Junio (columnas 3-5)
        - Q3: Julio-Septiembre (columnas 6-8)
        - Q4: Octubre-Diciembre (columnas 9-11)

    Args:
        data: Matriz de datos mensuales (años x 12 meses)

    Returns:
        Matriz de datos trimestrales (años x 4 trimestres)

    Examples:
        >>> data = np.array([[1,2,3,4,5,6,7,8,9,10,11,12]])
        >>> quarters = by_quarters(data)
        >>> quarters[0].tolist()
        [6, 15, 24, 33]
    """

    # Cortamos y sumamos por bloques de columnas
    q1 = np.sum(data[:, 0:3], axis=1) # Ene-Mar
    q2 = np.sum(data[:, 3:6], axis=1) # Abr-Jun
    q3 = np.sum(data[:, 6:9], axis=1) # Jul-Sep
    q4 = np.sum(data[:, 9:12], axis=1) # Oct-Dic

    # Apilamos las 4 columnas nuevas una al lado de otra
    # column_stack es muy común para "pegar" columnas
    return np.column_stack((q1, q2, q3, q4))
```

```
In [108... def check_by_quarters():
    global employment_data

    ym_data2 = \
        np.array([[ 9, 10,  9, 10,  3, 20, 17, 10,  7,  7, 23, 15],\
                  [12, 17, 15, 11, 11, 13, 15,  4, 12, 24, 14, 17],\
                  [17, 11, 21, 13, 19, 13, 16, 12, 10, 16, 15, 17]])
    quarters = by_quarters(ym_data2)
    print(f"Triemestres-1:\n {quarters}")
    print("\n=====")
    ym_data, ini_y = year_month_data(employment_data)
    quarters = by_quarters(ym_data)
    print(f"Triemestres-2:\n {quarters}")
```

```
check_by_quarters()
```

Triemestres-1:

```
[[28 33 34 45]
 [44 35 31 55]
 [49 45 38 48]]
```

=====

Triemestres-2:

```
[[4317 3894 3291 4443]
 [4021 3847 2820 4195]
 [4714 4027 3201 4787]
 [3523 2961 3280 5346]
 [4298 5419 2955 3473]
 [5190 5418 4560 6031]
 [5224 4273 3515 4781]
 [5495 6682 3542 3967]
 [3120 8587 940 0]]
```

Salida esperada:

```
Triemestres-1:
[[28 33 34 45]
 [44 35 31 55]
 [49 45 38 48]]
```

=====

```
Triemestres-2:
[[4317 3894 3291 4443]
 [4021 3847 2820 4195]
 [4714 4027 3201 4787]
 [3523 2961 3280 5346]
 [4298 5419 2955 3473]
 [5190 5418 4560 6031]
 [5224 4273 3515 4781]
 [5495 6682 3542 3967]
 [3120 8587 940 0]]
```

Comprobación de docstring

In [109... `help(by_quarters)`

Help on function by_quarters in module __main__:

`by_quarters(data: <built-in function array>) -> <built-in function array>`

Agrupar los datos mensuales en trimestres: 12 meses -> 4 trimestres

- Q1: Enero-Marzo (columnas 0-2)
- Q2: Abril-Junio (columnas 3-5)
- Q3: Julio-Septiembre (columnas 6-8)
- Q4: Octubre-Diciembre (columnas 9-11)

Args:

data: Matriz de datos mensuales (años x 12 meses)

Returns:

Matriz de datos trimestrales (años x 4 trimestres)

Examples:

```
>>> data = np.array([[1,2,3,4,5,6,7,8,9,10,11,12]])
>>> quarters = by_quarters(data)
>>> quarters[0].tolist()
[6, 15, 24, 33]
```

Comprobación de doctests

```
In [110...] doctest.run_docstring_examples(by_quarters, globals(), verbose=True)
```

Finding tests in NoName

Trying:

```
data = np.array([[1,2,3,4,5,6,7,8,9,10,11,12]])
```

Expecting nothing

ok

Trying:

```
quarters = by_quarters(data)
```

Expecting nothing

ok

Trying:

```
quarters[0].tolist()
```

Expecting:

```
[6, 15, 24, 33]
```

ok

Corrección automática

```
In [111...] testing.test_by_quarters(by_quarters)
```

```
data: [[ 9 10  9 10  3 20 17 10  7  7 23 15]
```

```
[12 17 15 11 11 13 15  4 12 24 14 17]
```

```
[17 11 21 13 19 13 16 12 10 16 15 17]]
```

by_quarters OK

```
In [112...] testing.test_by_quarters_typehints(by_quarters)
```

by_quarters type hints OK

```
In [113...] testing.test_docstring(by_quarters)
```

docstring OK

```
In [114...] testing.test_doctests(by_quarters, globals())
```

doctests OK